

Ochorus Engineering Audit

A full-stack review of the Ochorus reader — the Django/DRF API, the SvelteKit static SPA and PWA, the fixture-and-seed content pipeline, and the Render/Supabase deployment. Five independent review passes covered the backend, frontend,

content tooling, architecture, and the API/sync layer; every finding below was verified against the actual source, with deliberate, documented design decisions excluded.

19 shipped across two days of fixes · 5 in progress / paused on you · 67 open. Every finding below carries its status; shipped ones cite the PR.

30 significant bugs & defects 10 shipped · 1 in progress · 20 open

Ordered by severity. Each entry states the defect, the concrete failure it produces, and why it needs fixing. The most serious cluster is user-data loss in the reading-sync layer and content-destruction paths in the importers.

- 1

✓ #1231

Cross-device highlight loss: the marks PUT is a blind full-replace

HIGH

`MarksView.put` replaces a chapter's entire mark list with whatever the client sends, and the client pushes its full local list. Device B, whose cache predates device A's new highlight, pushes and erases it server-side; when A signs out, its local copy is wiped too — the highlight is permanently gone. **Why fix:** silent, permanent loss of the app's signature user data; the merge endpoint already unions correctly, so the fix pattern exists in the same file.

backend/reading/views.py:283-319 · frontend/src/lib/readingSync.ts:154-164
- 2

✓ #1229

Re-importing sermons resurrects a deliberately unpublished sermon

HIGH

`import_sermons` puts `is_published: True` in its upsert `defaults`, violating the repo's own create-only convention (which `seed_sermons` and `import_ochorus` carefully follow). A sermon pulled in prod for a copyright complaint is silently republished by the next re-import. **Why fix:** this is a policy/legal walk-back with no visible symptom until the complainant notices.

backend/library/management/commands/import_sermons.py:344-358
- 3

✓ #1230

A transient fetch failure during CCEL re-import silently deletes chapters and shifts every order

HIGH

`import_ccel` catches a failed section fetch, warns, and continues; `upsert_book` then deletes all existing chapters and recreates only the survivors. One 503 mid-crawl shifts every later chapter's `order` — breaking plan days, readers' saved positions, and prerendered URLs; if every fetch fails, a published book is left with zero chapters. **Why fix:** chapter order is a load-bearing contract the seeds explicitly refuse to touch; `import_web` already shows the correct abort-on-first-failure pattern.

backend/library/management/commands/import_ccel.py:537-546 · library/ingest.py:444
- 4

✓ #1228

CSP connect-src wildcard defeats its own token-exfiltration defense

HIGH

The CSP allows `https://*.supabase.co`, and its own comments say `connect-src` is the control that stops an injected script posting the localStorage JWT to an attacker (since `script-src` keeps `'unsafe-inline'`). Anyone can create a free Supabase project and receive traffic on that wildcard — so any XSS exfiltrates the token, including the admin's. **Why fix:** pinning the single project ref is a one-line change that restores the one control the design leans on.

render.yaml:163 (policy rationale at 132-149)
- 5

✓ #1227

Django admin login exposed unthrottled on the public API origin

HIGH

`/admin/` is mounted with password auth while DRF throttles cover only search/click/reading; there is no logout, no 2FA, and only minimal password validators. A compromised superuser bypasses the whole Supabase/allowlist model and can rewrite content served to every reader. **Why fix:** it's an unbounded online brute-force surface duplicating an admin that already exists behind `IsAdminEmail`.

backend/config/urls.py:38 · config/settings.py:122-125, 169-182
- 6

✓ #1229

An empty chapterize result wipes an existing book's chapters while it stays published

HIGH

In `import_ochorus`, when `chapterize()` returns nothing for an existing book (PDF layout change, chapterizer regression), the upsert still deletes all chapters and recreates none — and since `is_published` is create-only, the book remains live with zero chapters. The "saved unpublished" message is only true for new books. **Why fix:** a routine re-import can blank a live book with a success-looking log line.

backend/library/management/commands/import_ochorus.py:559, 614-619
- 7

✓ #1252

Reading progress PUT is last-write-wins with no recency check

MEDIUM

The merge path compares `updated_at` and keeps the newer position, but the live PUT carries no timestamp: a backgrounded tab flushing a stale debounced push overwrites a newer position from another device, and "Continue reading" jumps backwards. **Why fix:** the recency rule exists precisely to prevent this and is applied on only one of the two write paths.

backend/reading/views.py:262-280 vs 526-530 · frontend/src/lib/readingSync.ts:137-152
- 8

✓ #1252

The concurrency race fixed for plan progress was left open for marks

MEDIUM

`_upsert_plan_progress` does create-then-lock-then-union with `select_for_update` (with a regression test proving it), but `_merge_marks` and `_merge_sermon_marks` do the same read-union-write with no lock. Two devices merging concurrently can drop marks the endpoint's docstring promises are never dropped. **Why fix:** it's the provably known-bad pattern, kept alive next to its own fix.

backend/reading/views.py:542-612 vs 163-211
- 9

✓ #1252

A missing timestamp skips the merge recency guard entirely

MEDIUM

In `_merge_progress`, a row with an unparseable or absent `updated_at` unconditionally overwrites the server row. An older client or corrupted localStorage bundle clobbers newer positions on sign-in — the exact loss the guard exists to prevent. **Why fix:** the safe default for an unknown age is to keep the server row, not replace it.

backend/reading/views.py:526-529

10 open **An all-invalid marks payload is treated as "delete everything" instead of a 400** MEDIUM

`_marks_from_payload` silently drops malformed entries; if every entry is malformed the emptied list takes the delete branch, wiping the chapter's server-side highlights and notes. **Why fix:** a client bug in payload shape should be rejected loudly, not converted into irreversible data deletion.

`backend/reading/views.py:299-305, 339-345`

11 open **Quote's one-of invariant has no DB constraint, and violating it 500s a public page** MEDIUM

"Exactly one of chapter/sermon" is enforced nowhere in the database, though the Supabase SQL editor is a documented second write path — and `QuotePageView` dereferences `q.chapter.book` whenever `sermon_id` is null, so a both-null reviewed row crashes the author's public quote page. **Why fix:** the repo's own rule is invariants-in-DB; a `CheckConstraint` plus a null-guard closes both halves.

`backend/library/models.py:638-651` · `library/views.py:784-786`

12 open **The DEBUG admin bypass trusts loopback and is false behind any tunnel** MEDIUM

The bypass keys on `REMOTE_ADDR` being loopback, but an ngrok/cloudflared tunnel or local nginx presents loopback as the peer — granting full mutating admin to any remote client of a DEBUG dev box. The docstring's claim that "a remote client never gets the bypass" is untrue in the most common way a dev box gets shared. **Why fix:** demoing a dev box through a tunnel is routine; the stated guarantee should be real.

`backend/accounts/permissions.py:27-46`

13 open **import_biographies permanently clobbers curated author bios** MEDIUM

It overwrites `Author.bio` unconditionally with a short scraped paragraph, and the self-heal path in `author_sync` only replaces empty or known-stub bios — so a scraped paragraph is neither, and the curated long-form bio never re-lands. It also uses bare `create()` with no transaction. **Why fix:** one run silently destroys every hand-curated bio, and it is the one destructive command with no dry-run.

`backend/library/management/commands/import_biographies.py:62-68` · `library/author_sync.py:139-143`

14 open **Edits to a topic's English title or description never ship** MEDIUM

`seed_topics` puts prose in `get_or_create` defaults and only refreshes scripture fields on existing rows — yet `topic_seed.py` is a declared content root, so editing a shelf description moves the content digest and rebuilds the reader against the API's old prose, forever. Translations *are* upserted, so the asymmetry is an oversight. **Why fix:** it's the documented "prose reached the API, moved no digest" failure class, inverted — and Topic has no admin editor as an alternative path.

`backend/library/management/commands/seed_topics.py:36-52` vs `72-92`

15 open **Rate limits are per-worker fiction, reset under load** MEDIUM

Throttle counters live in per-worker `LocMemCache` across 3 gunicorn workers (so every limit is effectively 3×), and `--max-requests 500` recycles workers — erasing all buckets — precisely when a scraper is hammering the API. **Why fix:** the assets these throttles protect (unauthenticated search logging, the quadratic fuzzy-suggest scan) are exactly what an abuser targets, and the actual bound is far weaker than the configured rates.

`backend/common/throttling.py:35` · `config/settings.py:332-339` · `backend/Dockerfile` (CMD)

16 paused **CI is not an enforced merge gate, and red code auto-deploys** MEDIUM

The workflow's own comments record a PR merged four minutes before its CI run finished, and `autoDeploy: true` ships main straight to production. `preDeployCommand` failure keeps the API up, but a runtime-only bug ships unimpeded. **Why fix:** with many parallel sessions merging to a fast-moving main, an outrunnable gate is a gate in name only — branch protection makes it structural.

`.github/workflows/ci.yml:3-6, 208-210` · `render.yaml`

17 ✓ #1254 **Admin content actions never rebuild the prerendered reader** MEDIUM

The web deploy hook is consumed only by language go-live. Import publishes, translation approvals, and quote reviews all mutate reader-visible state with no rebuild trigger, so prerendered book, author, and sitemap pages stay stale until an unrelated commit touches a buildFilter path. **Why fix:** half the write paths invalidate and half don't — crawlers and social previews serve stale HTML indefinitely for the admin-borne half.

`backend/library/goLive.py:47` (sole hook consumer) · `library/admin_import_views.py`

18 open **The deploy-ordering wait can fail and nothing retriggers the web build** MEDIUM

`await-api-release.mjs` polls for 5 minutes, but the API deploy is a full Docker build plus the whole `release` seed chain over a 97 MB fixture — plausibly longer on a cold cache. When the timeout hits, the web build fails and prerendered pages keep old content until a human manually redeploys. **Why fix:** failing loudly is the design, but a deploy-ordering hazard that ends in "wait for a human to notice" is only half-mitigated.

`frontend/scripts/await-api-release.mjs:54, 163-171`

19 open **The RLS "no grants" test runs in no automated environment** MEDIUM

`DataApiGrantTests` skips when the `anon / authenticated` roles don't exist — which is true in CI's plain `postgres:17-alpine`. RLS-*enabled* is enforced; the second lock (no privileges granted on the schema) is verified nowhere automated, so a regression re-granting privileges ships green. **Why fix:** a table reachable by the anon key is a data leak, and this defense currently rests on nobody making a mistake.

`backend/accounts/tests_rls.py:112-116` · `.github/workflows/ci.yml:43-47`

20 open **Every first-time visitor gets a spurious full-page reload** MEDIUM

The `controllerchange` listener unconditionally reloads, and the service worker's activate calls `clients.claim()` — on a first-ever install there is no old controller, so install → activate → claim fires the listener and reloads the page under a reader seconds after they started reading (also killing the "available offline" toast). **Why fix:** it hits 100% of first visits; the standard fix is skipping the reload when there was no controller at setup.

`frontend/src/lib/pwa.svelte.ts:57-61` · `src/service-worker.ts:110`

21 [open](#) **A stale scroll-save timer corrupts the next chapter's resume point on every device** [MEDIUM](#)

The chapter reader's 250 ms debounced scroll-save is not cleared on chapter change (the shared `Reader.svelte` clears its own, with a comment explaining exactly why). Navigating within the debounce window fires the timer against the new chapter's DOM before restore: the saved anchor is deleted and paragraph 0 is pushed to the account, losing the reader's place in that chapter everywhere. **Why fix:** it silently destroys synced positions on the app's most-used surface.

```
frontend/src/routes/books/[slug]/[order]/+page.svelte:661-669 vs
src/lib/components/Reader.svelte:135-137
```

22 [open](#) **"Download for offline" reports success even when chapters failed to cache** [MEDIUM](#)

Per-chapter failures are swallowed and never counted; the metadata records the full chapter count and `download()` returns true regardless. On a flaky connection, the UI says the book is downloaded and the reader discovers the holes only when offline — the exact situation the feature exists for. **Why fix:** an offline feature whose success signal can lie fails at its whole purpose.

```
frontend/src/lib/offlineBooks.svelte.ts:107-119, 163-173
```

23 [open](#) **Theme choice frequently never syncs across devices** [MEDIUM](#)

The debounced profile push tracks the *resolved* theme but sends the *preference* — so changing preference without changing the resolved appearance (system→dark on an OS-dark machine, dark→system, etc.) re-runs nothing and the PATCH never fires. No other path pushes theme. **Why fix:** a settings toggle that appears to work but silently doesn't persist is the kind of bug users blame on themselves.

```
frontend/src/routes/+layout.svelte:63-70 ·
src/lib/auth.svelte.ts:252
```

24 [open](#) **A Supabase auth blip silently turns every signed-in user anonymous** [MEDIUM](#)

The JWKS client is a per-process singleton with no negative caching or backoff; when the key fetch fails, every request's auth resolves to anonymous (by design, per-failure) — but during an outage that means reading state stops syncing and the admin is locked out while `/api/health/` stays green, plus every request pays a synchronous fetch attempt. **Why fix:** invisible degradation with no alarm is the worst failure mode; a short negative-cache and a health signal fix both halves.

```
backend/accounts/authentication.py:79-97,
180-199
```

25 [open](#) **The legacy HS256 path stays live as long as the shared secret exists** [MEDIUM](#)

The algorithm-confusion defense itself is correct (per-branch pinned algorithm sets), but the design accepts either an HS-signed or a JWKS-signed token at all times. After migrating to asymmetric keys, a leaked old shared secret still mints valid tokens — and admin gating trusts the token's `email / email_verified` claims. **Why fix:** retired credentials should stop working; a config flag to disable the legacy branch post-migration removes a standing skeleton key.

```
backend/accounts/authentication.py:126-132 ·
accounts/permissions.py:55
```

26 [open](#) **Fast navigation shows one sermon's scripture under another sermon's heading** [LOW-MED](#)

The effect fetching the "preaching text" card has no cancellation token, unlike the same pattern done correctly in the chapter reader and `scripture.svelte.ts`. Navigating sermon A → B quickly lets A's slower response resolve last and write its verse text under B's reference. **Why fix:** displaying the wrong scripture passage under a sermon's heading is a content-integrity bug, not just a flicker.

```
frontend/src/routes/sermons/[slug]/+page.svelte:144-151
```

27 [open](#) **Opaque error responses are cached permanently as book covers** [LOW-MED](#)

Cover fetches use `mode: 'no-cors'` and `cache.put` with no `res.ok` check (the chapter path checks); an opaque response can be a CDN 404/500 and lands in the durable cache that activate deliberately never clears, served first forever. **Why fix:** one CDN hiccup gives a downloaded book a permanently broken cover, online and offline, until re-downloaded.

```
frontend/src/lib/offlineBooks.svelte.ts:151-159
```

28 [open](#) **Scrolling ⇌ Page layout drops the reader to the chapter start** [LOW-MED](#)

The layout toggle only writes the preference; the paged effect re-measures but never maps the saved anchor to a page, and returning to scroll never re-runs restore. A reader on paragraph 40 lands on page 1, then back at the top — while the anchor sits in storage the whole time. **Why fix:** losing your place for touching a display setting erodes trust in the reader's position-keeping generally.

```
frontend/src/routes/books/[slug]/[order]/+page.svelte:421-436
```

29 [open](#) **The service worker can respond with `undefined` instead of a Response** [LOW-MED](#)

In `staleWhileRevalidate`, when nothing is cached and the fetch rejects, the catch resolves to the undefined cached value and `respondWith` throws a `TypeError` rather than returning `Response.error()` like the sibling paths (an `as Promise<Response>` cast hides the type lie). Two independent review passes found this. **Why fix:** the first offline visit to an unvisited resource should fail cleanly, and the cast masks the bug from the compiler.

```
frontend/src/service-worker.ts:250-260
```

30 [open](#) **Fixture readers decode with the host locale, in the deploy path** [LOW-MED](#)

Three `read_text()` calls in `content_fixtures.py` (plus one in `regen_fixture.py`) omit `encoding="utf-8"` while sibling lines in the same module pass it. Under a C/POSIX locale — a container without `LANG` — the deploy seed aborts on the first non-ASCII translation fixture. **Why fix:** it's drift, not policy, and it sits directly in the deploy chain where an environment change turns it into a failed release.

```
backend/library/content_fixtures.py:249, 269,
283 · scripts/regen_fixture.py:162
```

31 open Fallback readers get broken prefetch and mislabeled progress LOW-MED

The next-chapter prefetch is built with the requested UI locale rather than the resolved edition, so an English-fallback book under `/es` prefetches a 404 and caches nothing — the prefetch is a silent no-op for exactly the readers on slow paths. Progress likewise records `getLang()` while marks deliberately record the resolved language, so the two stores disagree about what was read. **Why fix:** when a real Arabic/Spanish edition later ships, resume points claim a text their paragraph indices were never measured against — the same mispainting the marks comment exists to prevent.

```
frontend/src/routes/books/[slug]/[order]/+page.svelte:351, 569-587 (cf. 696-702)
```

20 best-practice recommendations 6 shipped · 2 in progress · 12 open

Design and architecture practices the codebase should adopt (or finish adopting — several exist in one place and are missing in another). Each carries the reason it matters here specifically.

1 paused Make CI a required status check with branch protection

The workflow asks to be added as a required check but isn't enforced — a PR has merged before its own CI finished, and `autoDeploy` ships main to production. **Why:** with many parallel sessions merging to a fast-moving main, any gate that depends on discipline will be outrun again; enforcement must be structural.

2 ✓ #1228 Pin the CSP to the exact Supabase project, and drop `img-src https:`

`connect-src` is the design's stated defense against token exfiltration, and `img-src https:` separately allows GET-shaped beacons to any host from sanitized-but-`{@html}` content. **Why:** a security control that allows attacker-registrable origins provides the feeling of defense without the substance.

3 open Move throttle counters and the default cache to a shared backend

A Postgres cache table (Supabase is already there) or a small Redis makes limits real across workers and lets them survive worker recycling; it also gives server-side memoization one durable home. **Why:** every throttle and memo in the app currently silently assumes a single process that doesn't exist.

4 ✓ #1227 Disable or fence the Django admin in production

The real admin is the SPA dashboard behind `isAdminEmail`; `/admin/` duplicates it behind a weaker password model with no logout. **Why:** every parallel auth surface is attack surface; the least-used one with the weakest protections is the one that gets brute-forced.

5 open Validate write payloads with DRF serializers, not per-view hand-rolled helpers

The reading app's serializers are output-only; input coercion is re-implemented slightly differently per endpoint — the pattern that produced the marks-wipe, recency-skip, and error-shape bugs. **Why:** serializer validation gives uniform 400s with field errors and one place to get coercion right instead of five.

6 open Validate `?language=` once at the boundary against the Language registry

`language_from_request` returns the raw query param unbounded; junk flows into log rows, cache keys (a cache-fill vector), and admin analytics, and consumers hand-truncate inconsistently. **Why:** one `known_codes()` check ends three separate downstream problems and removes a class of forgotten-truncation bugs.

7 open Enforce every invariant in the database, per the repo's own rule

Quote's "exactly one of chapter/sermon" has no `CheckConstraint`; `cover_color` ("hex" per comment) has no validator and flows into CSS. **Why:** the Supabase SQL editor and scripts are documented second write paths — Python-only invariants don't bind them, and the repo already knows this (the rule is written down; it just isn't applied here).

8 ✓ #1252 Use one locked-union-upsert primitive for all per-user merge state

Plan progress got create-then-lock-then-union plus a `TransactionTestCase`; marks and sermon marks kept the racy read-union-write. **Why:** when a concurrency fix is made once and its siblings aren't migrated, the codebase documents the bug it still contains.

9 open Return one error envelope everywhere

`_dict_body`'s validation error renders as a bare JSON array, escaping the project's own exception handler, while every other error is `{"detail": ...}` — and the frontend documents that callers read `.detail`. **Why:** clients can only build reliable error UX against a shape they can rely on; the test asserting only status 400 makes the regression invisible.

10 open Give every compatibility shim a sunset mechanism

The `sermon_marks` shim (three code sites) self-describes "retire once old bundles age out" with no usage counter or ratchet test, and the frontend version-sniffs by checking response shapes. **Why:** shims with no retirement signal live forever by default, and each API evolution stacks another sniff on top — this is API versioning by accretion.

11 open Fail closed on security configuration

The admin allowlist defaults to a personal email committed to the repo; a deploy that forgets `ADMIN_EMAILS` silently grants admin to whoever controls that address — while the settings module already fail-fasts on other missing required config one screen down. **Why:** security config should follow the stricter of the two patterns the file already contains.

12 ✓ #1256 Add automated dependency vulnerability management

No Dependabot/Renovate, no `pip-audit` / `npm audit` in CI. Lockfiles are frozen (reproducible — good), which means a disclosed CVE in Django, PyJWT, or supabase-js sits unpatched until a human notices. **Why:** for a public site handling auth tokens, "frozen and unwatched" is the worst quadrant.

13 ✓ #1254 **Route every reader-visible mutation through one content-changed channel**

Bump a DB-backed content revision and fire the (debounced) web deploy hook from a single service used by publish, approve, go-live, and quote review alike. **Why:** the elaborate digest apparatus covers repo-borne content only; admin-borne content has no invalidation channel, and the revision row is also the missing piece for ETags.

14 open **Add a staging deploy gate**

Every merge goes straight to prod; the 6-hourly prerender audit exists because four production incidents were detectable only after deploy — a scheduled probe of prod doing a staging environment's job with up to six hours of exposure. **Why:** the cheapest version (deploy previews promoted on green) converts detection-after into prevention-before.

15 open **Adopt an expand/contract migration convention**

`release` migrates while the old code still serves; nothing documents backwards-compatibility requirements or tests old code against the new schema. **Why:** the window is small but unguarded — one column drop in a routine PR turns a zero-downtime deploy into an error window, and the convention costs only documentation plus habit.

16 ✓ #1230 **Importers: abort on fetch failure, and retry with backoff first**

`import_ccel` skips failed sections (shifting chapter orders); fetches are single-shot while the translation pipeline has a proper backoff ladder to copy. `import_web` already aborts correctly. **Why:** for content whose ordering is a load-bearing contract, partial success is worse than failure — the failure modes are silent and corrupting rather than loud and safe.

17 open **Single-source the create-only-upsert seed pattern**

The "create-only fields survive deploys" convention is hand-rolled in at least three seeds, and `import_sermons` is the copy that missed it — producing the sermon-resurrection bug. **Why:** a convention that must be re-implemented correctly each time will eventually be implemented incorrectly; a shared helper makes the rule executable.

18 open **Pass `encoding="utf-8"` explicitly on all fixture and content IO**

The fixture modules mix explicit-UTF-8 lines with locale-default lines in the same file, and the corpus is heavily non-ASCII. **Why:** the deploy seed's correctness should not depend on a container's `LANG`; one lint rule (or `rufl`'s `PLW1514`) enforces it forever.

19 open **Cover the sync path with end-to-end tests**

The e2e suite blocks service workers and never signs in, so merge/push/wipe — where all the confirmed data-loss bugs live — are covered only by unit tests on each side, never through the real client-server seam. **Why:** the riskiest data path in the product currently has zero integration coverage; one authenticated Playwright flow (highlight on A, merge on B) would have caught the top bug.

20 paused **Turn on the observability that's already wired**

Sentry is a no-op until a DSN is set, frontend source maps are deliberately not uploaded (browser errors unsymbolicated), and nothing monitors uptime beyond Render's health check. **Why:** several confirmed failure modes here are silent (auth degrading to anonymous, stale prerenders, failed offline downloads) — silent failures plus no telemetry means users are the monitoring.

20 refactor ideas 2 shipped · 0 in progress · 18 open

Structural changes that pay for themselves — most target duplication that has already drifted at least once, or seams where the confirmed bugs above live.

1 ✓ #1254 Unify content invalidation into one service

One "content changed" call that bumps a DB revision and debounce-fires the web deploy hook, used by every reader-visible mutation. **Why:** it fixes the prerender-staleness hole for admin actions and unblocks ETags in one move — `http_cache.py` already documents that a DB-side revision is the missing piece.

2 open Generate the render.yaml locale route blocks from project.inlang

~220 hand-maintained rewrite rules, nine per locale, each new locale a hand-edit plus manual Blueprint sync — and the missed-route failure class has already bitten `/topics` and `/search`. **Why:** `svelte.config.js` already reads the same source of truth; a generator plus a CI check retires the whole failure class.

3 open Give the duplicated digest algorithm a cross-language golden test

Python's `content_digest()` and JS's `localDigest()` must stay byte-identical by hand ("must stay in step", says the comment). **Why:** the deploy-ordering mechanism silently breaks the day they diverge; a shared fixture-vector test — or having the build read a backend-emitted digest file — makes the invariant enforced instead of remembered.

4 open Split the import/translation toolchain out of the production image

`anthropic`, `pymupdf`, `mammoth`, `lxml` ship in the prod API image although prod never holds an Anthropic key. **Why:** an optional dependency group shrinks image size, build time, and — more importantly — the RCE surface of the internet-facing service.

5 open Extract the reading-view validation preamble

`_profile` + `_dict_body` + `_valid_slug` + `_kind_or_none` repeats verbatim in five views. **Why:** a shared decorator is also the single place to fix the inconsistent 400 shape and to apply serializer-grade validation later — five copies is five chances to diverge, and they already have.

6 ✓ #1252 Generalize the locked union-upsert to marks

Lift `_upsert_plan_progress`'s create-lock-union into a helper used by `_merge_marks` and `_merge_sermon_marks`. **Why:** it deletes two of the three divergent merge implementations and fixes the concurrent-merge race as a byproduct rather than as a patch.

7 open Split `library/views.py` (1,002 lines)

The scripture trio and the quotes pair are self-contained clusters with heavy inline imports; admin modules meanwhile import an underscore-private helper from it. **Why:** the repo's own precedent — splitting the 1,190-line `admin_views` into a package — exists because god-modules resist safe change; this is the next one.

8 open Use `BOOK_CARD_ANNOTATIONS` where it's re-spelled

Two serializer sites inline `Count("chapters")`, `Sum(...)` instead of the constant whose docstring says it exists because this exact pair drifted once already. **Why:** when the annotation changes (say, excluding front matter from word counts), the inlined copies won't — and shelf sort and reading-time labels quietly disagree by page.

9 open Fold the two topic-membership implementations into one `_attach_books` / `_attach_sermons` in the view and `TopicListSerializer`'s fallback are two full implementations of "members present in this language, curated order" — and the fallback has already drifted once (a missing prefetch). **Why:** "same behavior, two homes" is the shape of every future topic bug.

10 open Break up `AdminReviewQueueView.get`

A ~100-line method doing rows, outcomes, notes, settled counts, facets, filtering, three sort modes, hand-rolled pagination, and flag attachment — with the detail view reaching in to borrow a private static method. **Why:** the enrichment pipeline and the filter/page logic can't be tested or reused separately, which is why the performance problems in it (see below) have nowhere clean to be fixed.

11 open Make `chapter_flags` the only implementation of its own rules

`AdminAuditView` re-implements every flag condition inline, importing only thresholds — the exact drift `backend/CLAUDE.md` forbids ("a second copy drifts — it already did once"). **Why:** a new flag added to `chapter_flags` silently never reaches the audit; restructuring it to return per-flag context lets both consumers share one truth.

12 open Extract the plan-index helpers from `serializers.py` (1,092 lines)

The module mixes serializers with pure query utilities (`plan_chapter_index`, `plan_book_index`, cover helpers) used directly by views. **Why:** it shrinks the import graph and mirrors the package split the project already values; serializer files that accrete query logic become the next god-module.

13 open Single-source `word_count`

Four commands inline `len(re.sub(r"<[^\>]+>", " ", html).split())` while `ingest.word_count` is the canonical copy. **Why:** word counts drive reading-time labels and shelf sort; four copies means counts can disagree depending on which command last touched a book.

14 open Merge the two book-upsert implementations

`ingest.upsert_book` and `import_ochorus.upsert` share the entire shape (create-only fields, delete-and-recreate chapters, corrections, word counts); they diverge exactly where the `chapter_title` invariant broke. **Why:** the divergence point of duplicated upserts is precisely where this audit found a real bug — collapsing them prevents the next one.

15 open Share the author-creation defaults

`seed_books` and `seed_sermons` carry the same defaults dict verbatim, including the same comments — the `original_language` / `is_imprint` lessons had to be patched into both. **Why:** history already shows every fix here is made twice or missed once.

16 open Extract the paging engine from the 1,212-line chapter reader

A `pagedLayout.svelte.ts` module owning `measurePages` / `goToPage` / `pageOf` / `anchor` mapping, with unit tests like the existing `pageMath.test.ts`. **Why:** three confirmed bugs (stale scroll timer, layout-toggle position loss, locale mismatch) live in this file's seams — the drift the shared `readerText` refactor stopped for text features is still live for position features.

17 open Collapse the duplicated position machinery

The chapter reader and `Reader.svelte` each implement `topVisibleIndex` / `restoreScroll` with different edge behavior — past the last paragraph one returns 0, the other the last index, so bookmark and listen-from-here target opposite ends of the chapter. **Why:** same contract, two answers; a shared position module makes the divergence structurally impossible.

18 open One persisted-reactive-collection base class

`favorites`, `planProgress`, `readingActivity`, and `offlineBooks` each hand-roll the identical ticks + read-per-call + `ochorus:sync` listener pattern. **Why**: four copies of subtle invalidation logic, and the base class is also the one place to add the parsed-JSON caching the performance section calls for.

19 open One `Drawer.svelte` with the one focus trap

`TocDrawer` hand-rolls a weaker focus trap (misses disabled/inert handling) while `SearchDrawer` uses the shared `focusTrap` action; both duplicate backdrop, panel shell, and Escape wiring. **Why**: accessibility behavior should not depend on which drawer you opened — shared chrome fixes the a11y drift structurally.

20 open Move search-page state transitions into a testable module

The 1,245-line search page keeps scope/type/sort transition logic inline; `searchState.ts` covers only the URL codec — and the confirmed scope-drop bug hid exactly in the untested inline transitions. **Why**: the codec's documented rule ("scope survives an empty query") can only be enforced by tests if the logic it governs is extractable.

20 performance ideas 1 shipped · 2 in progress · 17 open

Ordered roughly by leverage. The single biggest theme: the API origin has three concurrent request slots and no shared cache in front of it, so everything that reduces or absorbs origin work is multiplied.

1 paused Switch unicorn to threaded workers

Three sync workers means three concurrent requests, total — one slow request (the fuzzy-suggest vocabulary scan, an admin export) occupies a third of global capacity for up to 120 s, and prerender builds contend with real readers for the same slots. **Why**: gthread with `--threads 4` is a near-free ~4× for this IO-bound workload inside the same 512 MB.

2 paused Put a CDN/edge cache in front of the API

Catalog endpoints declare `public`, `max-age=60`, `stale-while-revalidate=600` but only individual browsers consume it — every reader hits the origin. **Why**: the headers are already correct; adding a shared cache (Cloudflare is already slated for the no-slash redirect) is the cheapest 10× capacity gain available and blunts the scraper risk wholesale.

3 ✓ #1261 Add a DB revision row and ship ETags/304s

`http_cache.py` documents exactly why ETags are absent (a digest-keyed tag would 304 forever past admin publishes) and names the fix; without it, large shelf payloads re-transfer in full every 60 s per client. **Why**: the same revision row also powers unified invalidation — one small table pays twice.

4 open Cache the scripture and quote endpoints

`ScriptureView` (hit on every reader popover click), `ScriptureGraphView` (walks the citation graph per request), `QuotePageView`, and `QuoteAuthorsView` are deterministic public data with neither `PublicContentCacheMixin` nor throttles — unlike every sibling catalog view. **Why**: the most-repeated runtime lookup in the product is currently the least cached.

5 open Memoize `TopicListView`

Every request rebuilds the entire topical universe — all topics, entries, sermon entries, plus fully-annotated card objects — while the language list (30 s) and popular searches (300 s) already got server-side memos. **Why**: it's the heaviest read endpoint without one, and the pattern to copy is in the same file.

6 open Stop the plan views double-executing their queriesets

`PlanDetailView.get_serializer_context` calls `get_object()` again after retrieve already did (row + annotation + days prefetch, twice); `PlanListView` runs `list(self.get_queryset())` right before the generic view executes the same queryset for the response. **Why**: both are silent 2× DB cost on every plan request, fixed by memoizing the object or building context in `retrieve`.

7 open Stream, don't materialize, in `AdminBookDetailView`

It pulls `body_text` and `body_html` of every chapter of every language edition into a list just to compute flags — tens of MB for a 36-chapter work in 8 languages, the identical shape as the documented 2026-08-14 OOM. **Why**: on a 512 MB instance this is the next OOM waiting; `.iterator()` keeps peak memory flat.

8 open Scope the review-queue queries

Every queue GET loads the entire `ReviewOutcome` and `TranslationNote` tables into Python (1,570+ notes get per-work reference lists built), and every bulk POST adds two more full scans. **Why**: the cost grows with total review history forever, paid on every page view of the busiest admin screen; filtering to the current page's languages/kinds is a straight win.

9 open Stop `_attach_flags` fetching whole books to compare tags

It concatenates the complete HTML of both editions of up to 25 books per queue page — every chapter body, twice per row — just to compare tag sequences. **Why**: potentially hundreds of MB of DB transfer per admin page view for a comparison that could run on stored digests or per-chapter streams.

10 open Aggregate the analytics in SQL

`_weekly_signups` fetches every profile's `created_at` and buckets in Python (grows with accounts forever); `_weekly_active` issues 8 queries whose `__date_gte` cast defeats `idx_progress_updated`; `_most_read` runs one finisher-count query per row. **Why**: `TruncWeek + Count` and one grouped query replace ~20 queries and unbounded row transfer with three aggregates.

11 `open` Collapse `ChapterDetailView` 's two queries into one

It looks up the Book only to pass the instance into the Chapter filter, which re-joins `book__author` anyway; filtering directly on `book__slug / book__language` is one query with identical 404 semantics. **Why:** this is the hottest read path in the product — every chapter open pays the extra round trip.

12 `open` Prefetch existing rows in `MergeView`

A worst-case merge drives ~8,000 sequential statements inside one transaction holding row locks (per-row `get_or_create` + `SELECT FOR UPDATE` + save for plans; per-row select + upsert for sermon marks) — and the 240/min throttle multiplies what one account can hold open. **Why:** `_merge_marks` already shows the batch-prefetch shape; applying it to the other two sections bounds transaction time and lock hold.

13 `open` Prefilter `apply_body_corrections`

Every deploy reads the whole corpus's chapter and sermon HTML to, in steady state, change nothing. **Why:** a DB-side prefilter (declared slugs, plus a LIKE for the hyphen rule) skips nearly every row and directly shortens every single deploy — the tax is paid on all releases, content or not.

14 `open` Bulk-fetch in `seed_books`

One `Author.get_or_create` and one `Book...first()` per fixture book means ~250 queries per deploy to confirm mostly nothing changed. **Why:** `in_bulk` by slug up front reduces it to two queries; deploy-chain time is user-visible (it delays the web build's release wait).

15 `open` Make `chapter_drift` compare cheaply first

The report-only drift diagnostic prefetches `body_html` for the whole library inside the atomic seed transaction. **Why:** comparing lengths or stored digests first avoids materializing the corpus in RAM on every deploy on a memory-tight instance — for output nobody reads on the happy path.

16 `open` Use `bulk_update` in `backfill_body_text`

On a fresh seed it issues ~3,000 single-row UPDATES, one round trip each. **Why:** chunked `bulk_update` cuts deploy-time round trips by orders of magnitude on the path every fresh environment and rebuild pays.

17 `open` Cache parsed `localStorage` until the next tick

`favorites.has()`, `planProgress.isDone`, and `marks.countFor` each re-parse the full JSON blob per call — a shelf of 60 cards parses the same favorites JSON 60 times per render tick, and the TOC does it per row. **Why:** the `ticks` invalidation already exists; caching the parsed object until it changes is a small change with felt scroll/render impact on shelves.

18 `open` Memoize the `i18n` key lookup

`t()` runs two regex replaces (`toSnake`) on every call, and it's called inside per-row derives — search results rebuild every row label on each keystroke. **Why:** a `Map` from key to message function is one line and removes regex work from the hottest reactive paths.

19 `open` Stop fetching whole catalogues for small widgets

Every sermon page fetches the full sermon catalogue (all fields, all summaries) to compute "related", unmemoized; the 404 page fetches the entire book and sermon catalogues to pick seven daily items. **Why:** a `?book=` filter and a small picks endpoint (or module-level memo) turn two of the most casual navigations from catalogue-sized payloads into trivial ones — on a 3-slot origin, these add up.

20 `open` Parallelize CI and relax worker recycling

CI runs the backend suite twice (SQLite, Postgres), a Docker build+boot, and the full frontend build+e2e serially in one job — the gate people outrun is partly a latency problem. Meanwhile `--max-requests 500` recycles each of 3 workers every ~500 requests, repaying startup, JWKS fetch, and cache warm (and zeroing throttles). **Why:** parallel jobs shorten the merge critical path without weakening the gate, and the memory ratchet the recycling guards against would be equally contained at 5,000.

Credit where due. The audit also confirmed unusual strengths worth preserving: per-branch JWT algorithm pinning with required claims, RLS enforced by a self-explaining test, N+1 discipline held by nine query-budget tests, sanitization enforced by corpus-scanning tests rather than hopeful `save()` hooks, the natural-key one-file-per-work fixture design, and a frontend that documents its own trade-offs in place. Most findings above sit in the seams between two correct implementations of the same idea — which is why so many of the refactors are "collapse the pair into one".

Produced by five independent verification passes over the working tree at commit 9e447f5d, August 28, 2026. Every finding cites the file and lines it was verified against; deliberate, documented design decisions were reviewed and excluded.